1 Decisões de implementação

1.1 Representação dos nós

Cada nó da árvore é um arquivo binário em `bin/`, nomeado `node_ID.bin`. O ID vem da variável global `nextId`, incrementada a cada nó criado, então nenhum nome se repete.

Em memória, `Node` guarda o vetor de chaves, a quantidade de chaves `n`, o indicador de folha, o nome do próprio arquivo e o vetor `children`. Esse vetor é `char **`: o filho é referenciado pelo nome do arquivo, não por ponteiro, porque endereços não sobrevivem ao fim da execução. A `BTree` guarda apenas o nome do arquivo da raiz e o valor de t.

Os demais nós só existem em memória durante a operação que os lê: cada função faz `diskRead`, usa o nó e libera com `freeLoadedNode`. Nas descidas (busca, inserção, remoção), o nome do filho é copiado e o pai é liberado antes da chamada recursiva, então em geral só o nó corrente fica carregado. As exceções são as operações locais (divisão, empréstimo e fusão, que mantêm pai, filho e irmão ao mesmo tempo), `removeFromInternal`, em que o nó permanece carregado durante a remoção na subárvore, e a impressão, que mantém o caminho da raiz até o nó lido.

1.2 Formato do arquivo de nó

`diskWrite` grava, nesta ordem: `n`, `leaf`, as $2t - 1$ posições do vetor de chaves (inclusive as não usadas) e, para cada uma das $2t$ posições de filho, um indicador de existência seguido, se existir, do tamanho do nome (já contando o `\0` final) e do nome. Como o vetor de chaves tem tamanho fixo, `diskRead` lê sempre a mesma quantidade, sem depender de `n`. Os nomes dos filhos são alocados com o tamanho lido, e as posições sem filho ficam nulas.

Erros de abertura, leitura e escrita são tratados como fatais: `die` mostra o motivo com `perror` e encerra o programa. O programa assume que os arquivos de nós só foram produzidos por `diskWrite`, então não valida o conteúdo lido (faixa de n, coerência de `leaf`). As chamadas a `remove`, que apagam nós descartados, não são verificadas: uma falha deixa apenas um arquivo órfão, sem afetar a árvore.

`diskWrite` grava em `<arquivo>.tmp` e depois substitui o original pela macro `REPLACE`, para que uma falha no meio de um `fwrite` não corrompa um nó válido. O retorno de cada `fwrite` e o de `fclose` são checados, já que erros de escrita em buffer só aparecem ali.

Essa técnica protege cada arquivo individualmente, mas não torna atômica uma operação que altera vários nós: se o programa for interrompido entre duas gravações, a

árvore pode ficar inconsistente. Para reduzir o dano, em `splitNode` o novo nó é gravado antes do pai, de modo que o pai nunca aponta para um arquivo inexistente; o nó original é o último a ser gravado.

1.3 Metadados e persistência

`bin/tree.meta` guarda t, `nextId` e o nome do arquivo da raiz, este precedido do seu tamanho (tamanho 0 indica árvore vazia). É gravado com a mesma técnica de arquivo temporário e `REPLACE`.

`loadMeta` devolve 1 quando carrega o arquivo, 0 quando ele não existe e -1 quando ele está inválido. A validação se resume a conferir que todos os campos puderam ser lidos por completo; t, `nextId` e a raiz só são copiados para `bt` e `nextId` depois que a leitura inteira tem sucesso. O programa não confere a faixa de t nem se o arquivo da raiz existe: se ele estiver faltando, o erro aparece no primeiro `diskRead` e é fatal.

Se o arquivo não existe, o programa trata como primeira execução e pergunta t. Se existe mas está inválido, `startTree` informa o problema e o programa encerra sem tocar nos arquivos, para não sobrescrever uma árvore recuperável.

Salvar `nextId` é necessário: sem ele, uma nova execução reiniciaria os IDs em 1 e sobrescreveria nós existentes. `saveMeta` roda ao fim de cada iteração do menu e ao sair, pois a raiz pode mudar em `insert` e `removeKey`. Os arquivos de nós não são apagados ao encerrar; `freeBTree` só libera o nome da raiz em memória.

1.4 Inserção

Usa a variante de passada única com divisão preventiva: todo filho cheio ($2t - 1$ chaves) é dividido *antes* de a descida entrar nele, então a inserção nunca precisa subir de volta. `insert` trata dois casos especiais: árvore vazia (cria uma folha como raiz) e raiz cheia (cria nova raiz interna com a antiga como único filho e chama `splitNode`, o que aumenta a altura em 1). O restante é feito por `insertNonFull`, que desce até a folha e insere deslocando as chaves maiores.

`splitNode` promove `keys[t-1]` ao pai, move as $t - 1$ últimas chaves (e, em nó interno, os t últimos filhos) para um nó novo e deixa as $t - 1$ primeiras no original. Os ponteiros movidos são zerados na origem para evitar *double free*.

1.5 Remoção

Também em passada única, com reforço preventivo: antes de descer para um filho com $t-1$ chaves, `fill` garante que ele tenha ao menos t, tentando nesta ordem emprestar do irmão esquerdo (`borrowFromPrev`), emprestar do direito (`borrowFromNext`) ou fundir

(`merge`). Se a fusão for com o irmão da esquerda, o filho passa a ser o de índice `idx-1`, e `removeFromNode` ajusta o índice.

Quando a chave está em nó interno, `removeFromInternal` trata três casos: filho esquerdo com ao menos t chaves (troca pelo predecessor e remove o predecessor da subárvore esquerda); filho direito com ao menos t chaves (sucessor, simétrico); ambos com $t-1$ (fusão e remoção da chave no nó fundido). Nos três casos o pai é gravado antes da descida, senão a troca da chave, ou a perda de uma chave e de um filho na fusão, não chegaria ao disco. `merge` apaga o arquivo do irmão absorvido.

Ao final, `removeKey` confere a raiz: se ficou com 0 chaves e é interna, o único filho vira raiz (altura diminui em 1); se era folha, a árvore fica vazia e a raiz vira `NULL`. Em ambos os casos o arquivo da raiz antiga é removido.

`removeKey` é uma função `void`: não informa se a chave existia, e uma chave ausente simplesmente não altera a árvore. Por isso o menu chama `search` antes, para distinguir "não encontrado" de "removido".

1.6 Chaves duplicadas

Chaves repetidas são aceitas, então a ordem dentro dos nós é não decrescente. Na descida, a comparação para na primeira chave $\geq$ à procurada, então valores iguais seguem pelo filho da esquerda. A busca devolve o primeiro nó do caminho que contém a chave, e cada remoção elimina uma única ocorrência.

1.7 Impressão

`printBTree` lê a árvore uma única vez e monta uma árvore auxiliar de layout (`LNode`) com texto, largura e filhos de cada nó. Calcula a largura de cada subárvore de baixo para cima e a coluna de cada nó de cima para baixo, desenha nós e conectores (`/`, `|`, `\`) em uma grade de caracteres e imprime por níveis. A estrutura auxiliar é liberada ao fim e não altera os arquivos.

1.8 Entrada de dados

`readInt` lê um inteiro e descarta o resto da linha, para que sobras do buffer não sejam lidas como a entrada seguinte. Devolve 1 se leu, 0 se a entrada era inválida (mostra *Invalid input*) e -1 no fim da entrada (EOF). Como o menu limpa a tela a cada volta, toda entrada ou opção inválida é seguida de uma pausa até o ENTER; sem ela, a mensagem seria apagada antes de ser lida. O fim da entrada encerra o programa depois de salvar a árvore, pois sem esse tratamento os laços de leitura repetiriam a falha indefinidamente.

1.9 Compatibilidade Linux e Windows

O código compila nos dois sistemas. As diferenças ficam isoladas em macros no início do arquivo, escolhidas por `_WIN32`:

Operação	**Linux**	**Windows**
Criar `bin/` (`MKDIR`)	`mkdir(p, 0755)`	`_mkdir(p)`
Limpar a tela (`CLEAR`)	`system("clear")`	`system("cls")`
Substituir arquivo (`REPLACE`)	`rename`	`MoveFileExA`

A diferença relevante é a substituição de arquivo. No Linux, `rename` sobrescreve o destino; no Windows ela falha se o destino já existe, o que quebraria a técnica do arquivo temporário. Por isso o Windows usa `MoveFileEx` com `MOVEFILE_REPLACE_EXISTING`, e a macro inverte o retorno para que, nos dois casos, valor diferente de zero signifique falha. Os arquivos são abertos em modo binário (`"rb"`/`"wb"`), então o Windows não converte fins de linha dentro dos nós. Se `bin/` já existir, `MKDIR` falha sem consequência.

Duas limitações: o formato usa os tamanhos nativos de `int`, `long` e `size_t` (por exemplo, `long` tem 4 bytes no Windows e 8 no Linux de 64 bits), então o diretório `bin/` não deve ser copiado entre sistemas; cada plataforma gera o seu. E o texto "Nível" da impressão usa acento, que o terminal do Windows pode exibir incorretamente conforme a página de código (`chcp 65001` resolve).

2 Dinâmica do grau t

O valor de t define os limites de todos os nós:

	Mínimo	**Máximo**
Chaves por nó (exceto raiz)	$t-1$	$2t-1$
Filhos por nó interno	t	$2t$

Exemplos: com $t = 2$ o nó guarda de 1 a 3 chaves; com $t = 3$, de 2 a 5.

t é pedido ao usuário na primeira execução e aceito entre 2 e 1000. O mínimo 2 vem da própria definição (com $t = 1$ o nó poderia ter 0 chaves). O máximo é um limite prático da implementação, para impedir que um valor absurdo gere alocações enormes; não é propriedade da árvore B. Depois de definido, t é salvo em `tree.meta` e não pode mais ser alterado. Isso é intencional, pois o formato dos arquivos depende de t (cada nó tem $2t-1$ chaves e $2t$ posições de filho em tamanho fixo), e ler os nós com outro t os interpretaria de forma errada. Para usar outro t, é preciso apagar o diretório `bin/`.

A capacidade dos nós não é fixa no código: `createNode` e `diskRead` alocam `keys` com $2t-1$ posições e `children` com $2t$ (este com `calloc`, deixando os ponteiros nulos),

e `diskWrite` e `diskRead` percorrem essas quantidades. Os limiares de cada operação também são expressos em função de t: nó cheio é $n = 2t - 1$ (gatilho de `splitNode`); nó que precisa de reforço é $n < t$ (gatilho de `fill`); irmão com chaves de sobra é $n \geq t$ (empréstimo); a fusão ocorre quando os dois nós têm $t - 1$ e resulta em $2t - 1$ chaves.

Com n chaves, a altura h satisfaz $h \leq \log_t \frac{n+1}{2}$, então um t maior gera árvores mais baixas e menos acessos a disco por operação. Busca, inserção e remoção percorrem um único caminho da raiz até uma folha, sem voltar a nós já visitados, e fazem $O(h) = O(\log_t n)$ leituras e gravações de nós. Em inserção e remoção, cada nível lê o filho uma vez para verificar se está cheio (ou com $t - 1$ chaves) e outra vez ao descer para ele, o que dá cerca de duas leituras por nível; a ordem assintótica não muda. A remoção em nó interno acrescenta uma descida para obter o predecessor ou o sucessor, e o menu faz uma busca antes de remover, ambas também $O(h)$. Dentro de cada nó as chaves são percorridas linearmente, $O(t)$, custo de CPU pequeno perto do acesso a disco.

3 Testes realizados

Os testes foram feitos pelo menu do programa, inserindo valores e conferindo a saída de `printBTree` após cada operação. Em cada caso, verificou-se: (1) chaves em ordem não decrescente dentro de cada nó e entre nós; (2) nós (exceto raiz) entre $t - 1$ e $2t - 1$ chaves; (3) nós internos com $n + 1$ filhos; (4) todas as folhas no mesmo nível.

Caso	**Entrada**	**Verificado**
Divisão de folha e da raiz	Inserir chaves até a raiz atingir $2t - 1$	Nova raiz com a chave central; altura +1
Divisão de nó interno	Inserção suficiente para ter 3 níveis	Chave promovida corretamente; propriedades 1 a 4
Ordens de inserção	Sequência crescente, decrescente e aleatória	Mesmas propriedades nos três casos
Busca	Chave presente, ausente e repetida	Nó correto exibido; "not found" sem alterar a árvore
Remoção em folha	Chave em folha com mais de $t - 1$ chaves	Chaves deslocadas, n atualizado
Remoção em nó interno	Casos do predecessor, sucessor e fusão	Chave substituída ou fundida corretamente
Empréstimo	Filho com $t - 1$ e irmão com $\geq t$ (esq. e dir.)	Chave do pai desce, a do irmão sobe
Fusão	Filho e irmão com $t - 1$	Arquivo do irmão apagado; pai perde chave e filho
Redução de altura	Remover até a raiz ficar vazia	Filho vira raiz; no fim, árvore vazia
Chaves duplicadas	Inserir o mesmo valor várias vezes e remover	Uma ocorrência removida por chamada
Persistência	Encerrar e reabrir o programa	t e raiz recuperados; buscas ainda acham os valores; novos nós não reutilizam IDs
Valores de t	Repetir os casos acima com $t = 2$, $t = 3$ e outro valor maior	Limiares corretos para cada t
Entradas inválidas	$t < 2$, $t > 1000$, texto no lugar de número em qualquer menu, opção inexistente, fim da entrada (Ctrl+D)	Mensagem de erro e volta ao menu após ENTER; o fim da entrada encerra o programa salvando a árvore

Tabela 1: Casos de teste.